

Lab Report: Full Observability and Security Solution

Metrics, dashboards, alerting, centralized logging and account threat detection for a containerized Flask service on AWS

Prepared by: Celse Mizeromahire

Overview

A Flask weather service already deployed to EC2 by a Jenkins pipeline was given a monitoring and security stack. The pipeline proved the application could be built and shipped, but said nothing about whether the running system was healthy. This project adds metrics, dashboards, alerting, centralised logging, account audit and threat detection, provisioned with Terraform and configured with Ansible so the stack rebuilds from an empty account with no manual steps.

How the signals flow

One monitoring EC2 instance was added to the existing VPC, running Prometheus, Grafana and Alertmanager as a single Docker Compose project. It locates the VPC by tag rather than by hardcoded identifier, keeping the two Terraform stacks independent.

Prometheus pulls from each target every fifteen seconds, so security groups must allow the monitoring host to reach the exporter ports inside the VPC, and a target reported as down is more often a network rule than a failed exporter. Logs travel the other way: container output is pushed to CloudWatch Logs by the Docker `awslogs` driver, and Jenkins, a systemd service, by the CloudWatch agent. Both authenticate through an IAM instance profile, so no long-lived keys exist on either host.

What was built

Capability	Component	Configuration
Metrics	Prometheus 3.1	Six targets across four jobs, scraped every 15s, 15-day retention. Addresses rendered from the EC2 dynamic inventory, so no IP is written by hand.
Instrumentation	Flask exporter, node_exporter	RED metrics from a status-labelled request counter and a duration histogram; node_exporter on all three hosts.
Dashboards	Grafana 11.5	Application error rate, throughput and latency percentiles; host CPU, memory and disk. Dashboard and data source provisioned from repository files.
Alerting	4 rules, Alertmanager 0.28	Error rate, latency, application and host availability. Grouped by name and severity, routed to Discord with resolution notices.
Logging	CloudWatch Logs	Six log groups with 14-day retention declared in Terraform, not left to the log driver, which creates groups that never expire.
Audit and detection	CloudTrail, GuardDuty	Multi-region trail with log file validation into an encrypted, versioned bucket with tiered lifecycle rules; GuardDuty on CloudTrail, VPC flow and DNS logs, publishing every 15 minutes.

Alerting design

The requirement was an alert above a five per cent error rate. The application reports only a counter labelled by status, so Prometheus derives the ratio at query time: the five-minute rate of 5xx responses over the five-minute rate of all requests. A minimum request rate guards the denominator against very low traffic, and the rule holds for two minutes before firing.

The remaining rules cover 95th-percentile latency and failure to answer scrapes. Their scopes do not overlap, so one incident produces one notification, and because Alertmanager routes separately from evaluation an alert can be silenced during planned work without editing its rule.

Security posture

Prometheus has no authentication of its own, so access is controlled at the network layer: the security group admits only the operator's IP, and Grafana sits behind the same restriction with its own login. The metrics endpoint is limited to the VPC CIDR, and both hosts reach CloudWatch through an IAM instance profile, so no long-lived key exists to leak.

The latency panel, broken down by request path, showed sustained probing for credential and shell-history files within hours of the host becoming publicly reachable, every request answered with a 404. The GuardDuty findings in the evidence are samples, generated to exercise the detection path.

Verification

Signal	Result
Metrics and dashboards	All six targets healthy, confirming the scrape path across security groups, exporters and the application endpoint; dashboards populated under live traffic.
Alerting	Four rules loaded and evaluating; one observed moving from pending to firing under induced load, delivered to Discord and cleared on recovery.
Logs and security	Log lines confirmed in the application and Jenkins log groups; CloudTrail digest files present in the bucket, and GuardDuty findings displayed with severity and affected resource.

Conclusion

Every requirement is met and defined in code, so the stack rebuilds from an empty account and arrives in the same state. Three gaps remain: TLS on the published ports, Grafana's password in Secrets Manager rather than an Ansible variable, and remote write for retention beyond fifteen days.

Repositories

Observability platform: github.com/Smiley2507/monitoring-and-security-lab

Application and pipeline: github.com/Smiley2507/jenkins-cicd-lab